```
# ============================================================
# EXPERIMENT NO. 3
# Statistical Analysis and Hypothesis Testing using Python
# Dataset: Titanic Survival Dataset
# ============================================================


# ------------------------------------------------------------
# 1. IMPORT REQUIRED LIBRARIES
# ------------------------------------------------------------

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats
from scipy.stats import ttest_ind, f_oneway, pearsonr

import statsmodels.api as sm

from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

print("Libraries imported successfully!")
print("=" * 70)


# ------------------------------------------------------------
# 2. LOAD DATASET
# ------------------------------------------------------------

df = pd.read_csv("Titanic-Dataset.csv")

print("Dataset loaded successfully!")
print("=" * 70)


# ------------------------------------------------------------
# 3. BASIC DATA EXPLORATION
# ------------------------------------------------------------

print("\nFIRST 5 ROWS:")
display(df.head())

print("\nDATASET SHAPE:")
print(df.shape)

print("\nCOLUMN NAMES:")
print(df.columns.tolist())

print("\nDATASET INFORMATION:")
df.info()


# ------------------------------------------------------------
# 4. HANDLE MISSING VALUES
# ------------------------------------------------------------

df["Age"] = df["Age"].fillna(df["Age"].median())

df["Fare"] = df["Fare"].fillna(df["Fare"].median())

print("\nMissing values after treatment:")
print(df.isnull().sum())


# ============================================================
# DESCRIPTIVE STATISTICS
# ============================================================

# ------------------------------------------------------------
# 5. MEAN
# ------------------------------------------------------------

print("\nMEAN:")
print(df[["Age", "Fare"]].mean())


# ------------------------------------------------------------
# 6. MEDIAN
```

```
# ------------------------------------------------------------

print("\nMEDIAN:")
print(df[["Age", "Fare"]].median())


# ------------------------------------------------------------
# 7. MODE
# ------------------------------------------------------------

print("\nMODE:")
print(df[["Age", "Fare"]].mode().iloc[0])


# ------------------------------------------------------------
# 8. VARIANCE
# ------------------------------------------------------------

print("\nVARIANCE:")
print(df[["Age", "Fare"]].var())


# ------------------------------------------------------------
# 9. STANDARD DEVIATION
# ------------------------------------------------------------

print("\nSTANDARD DEVIATION:")
print(df[["Age", "Fare"]].std())


# ------------------------------------------------------------
# 10. COMPLETE DESCRIPTIVE STATISTICS
# ------------------------------------------------------------

print("\nCOMPLETE DESCRIPTIVE STATISTICS:")
display(df[["Age", "Fare"]].describe())


# ============================================================
# PEARSON CORRELATION
# ============================================================

# ------------------------------------------------------------
# 11. PEARSON CORRELATION BETWEEN AGE AND FARE
# ------------------------------------------------------------

correlation, correlation_pvalue = pearsonr(
    df["Age"],
    df["Fare"]
)

print("\nPEARSON CORRELATION")
print("Correlation coefficient:", correlation)
print("P-value:", correlation_pvalue)


# ------------------------------------------------------------
# 12. CORRELATION MATRIX
# ------------------------------------------------------------

correlation_matrix = df[
    ["Age", "Fare", "SibSp", "Parch", "Pclass", "Survived"]
].corr()

print("\nCORRELATION MATRIX:")
display(correlation_matrix)


# ------------------------------------------------------------
# 13. CORRELATION HEATMAP
# ------------------------------------------------------------

plt.figure(figsize=(9, 6))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix Heatmap")

plt.show()
```

```
plt.show()


# ============================================================
# HYPOTHESIS TESTING - INDEPENDENT SAMPLE T-TEST
# ============================================================

# Business Question:
# Is there a significant difference in Fare between
# passengers who survived and passengers who did not survive?


# ------------------------------------------------------------
# 14. CREATE TWO GROUPS
# ------------------------------------------------------------

survived_fare = df[df["Survived"] == 1]["Fare"]

not_survived_fare = df[df["Survived"] == 0]["Fare"]


print("\nAVERAGE FARE OF SURVIVORS:")
print(survived_fare.mean())

print("\nAVERAGE FARE OF NON-SURVIVORS:")
print(not_survived_fare.mean())


# ------------------------------------------------------------
# 15. FORMULATE HYPOTHESES
# ------------------------------------------------------------

# H0 (Null Hypothesis):
# There is NO significant difference in average fare
# between survivors and non-survivors.
#
# H1 (Alternative Hypothesis):
# There IS a significant difference in average fare
# between survivors and non-survivors.


# ------------------------------------------------------------
# 16. INDEPENDENT SAMPLE T-TEST
# ------------------------------------------------------------

t_statistic, t_pvalue = ttest_ind(
    survived_fare,
    not_survived_fare,
    equal_var=False
)

print("\nINDEPENDENT SAMPLE T-TEST")
print("T-statistic:", t_statistic)
print("P-value:", t_pvalue)


# ------------------------------------------------------------
# 17. INTERPRET T-TEST
# ------------------------------------------------------------

alpha = 0.05

if t_pvalue < alpha:
    print("\nDecision: Reject H0")
    print("There is a statistically significant difference.")
else:
    print("\nDecision: Fail to reject H0")
    print("There is no statistically significant difference.")


# ============================================================
# ONE-WAY ANOVA
# ============================================================

# Business Question:
# Is there a significant difference in average Fare
# among different passenger classes?


# ------------------------------------------------------------
# 18. CREATE GROUPS
# ------------------------------------------------------------

class1 = df[df["Pclass"] == 1]["Fare"]
```

```
class2 = df[df["Pclass"] == 2]["Fare"]

class3 = df[df["Pclass"] == 3]["Fare"]


print("\nAVERAGE FARE BY PASSENGER CLASS:")

print("Class 1:", class1.mean())

print("Class 2:", class2.mean())

print("Class 3:", class3.mean())


# ------------------------------------------------------------
# 19. FORMULATE ANOVA HYPOTHESES
# ------------------------------------------------------------

# H0:
# There is no significant difference in average fare
# among the three passenger classes.
#
# H1:
# At least one passenger class has a significantly
# different average fare.


# ------------------------------------------------------------
# 20. ONE-WAY ANOVA
# ------------------------------------------------------------

anova_statistic, anova_pvalue = f_oneway(
    class1,
    class2,
    class3
)

print("\nONE-WAY ANOVA")
print("F-statistic:", anova_statistic)
print("P-value:", anova_pvalue)


# ------------------------------------------------------------
# 21. INTERPRET ANOVA
# ------------------------------------------------------------

if anova_pvalue < alpha:
    print("\nDecision: Reject H0")
    print("There is a statistically significant difference")
    print("in average fare among passenger classes.")
else:
    print("\nDecision: Fail to reject H0")
    print("There is no statistically significant difference.")


# ------------------------------------------------------------
# 22. ANOVA VISUALIZATION
# ------------------------------------------------------------

plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="Pclass",
    y="Fare"
)

plt.title("Fare Distribution by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Fare")

plt.show()


# ============================================================
# LINEAR REGRESSION
# ============================================================

# Goal:
# Analyze relationship between Age and Fare.
```

```
# ------------------------------------------------------------
# 23. PREPARE DATA
# ------------------------------------------------------------

X = df[["Age"]]

y = df["Fare"]


# ------------------------------------------------------------
# 24. CREATE REGRESSION MODEL
# ------------------------------------------------------------

model = LinearRegression()

model.fit(X, y)


# ------------------------------------------------------------
# 25. PREDICT FARE
# ------------------------------------------------------------

y_pred = model.predict(X)


# ------------------------------------------------------------
# 26. REGRESSION COEFFICIENTS
# ------------------------------------------------------------

print("\nLINEAR REGRESSION RESULTS")

print("Intercept:", model.intercept_)

print("Coefficient:", model.coef_[0])


# ------------------------------------------------------------
# 27. R-SQUARED
# ------------------------------------------------------------

r2 = r2_score(y, y_pred)

print("R² Score:", r2)


# ------------------------------------------------------------
# 28. REGRESSION VISUALIZATION
# ------------------------------------------------------------

plt.figure(figsize=(9, 6))

sns.scatterplot(
    x=df["Age"],
    y=df["Fare"]
)

plt.plot(
    df["Age"],
    y_pred
)

plt.title("Linear Regression: Age vs Fare")

plt.xlabel("Age")

plt.ylabel("Fare")

plt.show()


# ============================================================
# STATSMODELS REGRESSION SUMMARY
# ============================================================

# ------------------------------------------------------------
# 29. STATSMODELS OLS REGRESSION
# ------------------------------------------------------------

X_stats = sm.add_constant(X)

ols_model = sm.OLS(y, X_stats).fit()

print("\nSTATSMODELS REGRESSION SUMMARY:")
print(ols_model.summary())
```

```
# ============================================================
# FINAL SUMMARY
# ============================================================

print("\n" + "=" * 70)
print("STATISTICAL ANALYSIS SUMMARY")
print("=" * 70)

print("\nPearson Correlation:", correlation)

print("T-Test P-value:", t_pvalue)

print("ANOVA P-value:", anova_pvalue)

print("Linear Regression R²:", r2)

print("\nStatistical analysis completed successfully!")


# ============================================================
# QUESTIONS AND ANSWERS
# ============================================================


# Q1. What is the difference between descriptive statistics
# and inferential statistics?
#
# Answer:
# Descriptive statistics summarizes and describes the data
# using mean, median, mode, variance and standard deviation.
#
# Inferential statistics uses sample data to make conclusions
# or predictions about a larger population using techniques
# such as hypothesis testing and confidence intervals.


# Q2. Explain the concepts of the Null Hypothesis (H0)
# and Alternative Hypothesis (H1).
#
# Answer:
# Null Hypothesis (H0) states that there is no significant
# difference or relationship between variables.
#
# Alternative Hypothesis (H1) states that a significant
# difference or relationship exists.
#
# Example:
# H0: There is no difference in average fare between
# survivors and non-survivors.
#
# H1: There is a difference in average fare between
# survivors and non-survivors.


# Q3. What is a p-value? How is it used to make
# statistical decisions?
#
# Answer:
# A p-value indicates the probability of observing the result
# if the null hypothesis is true.
#
# If p-value < 0.05:
# Reject H0.
#
# If p-value >= 0.05:
# Fail to reject H0.


# Q4. Differentiate between a t-test and ANOVA.
# In which situations is each test applied?
#
# Answer:
# A t-test is generally used to compare the means of two groups.
#
# ANOVA is used to compare the means of three or more groups.
#
# Example:
# T-test -> Survivors vs Non-survivors.
#
# ANOVA -> Fare among Class 1, Class 2 and Class 3.
```

```
# Q5. What does the Pearson correlation coefficient indicate?
# What are its possible values?
#
# Answer:
# Pearson correlation measures the strength and direction of
# a linear relationship between two numerical variables.
#
# Its value ranges from -1 to +1.
#
# +1 = Perfect positive correlation.
# -1 = Perfect negative correlation.
#  0 = No linear correlation.


# Q6. Explain the significance of R² (Coefficient of
# Determination) in Linear Regression.
#
# Answer:
# R² measures how much variation in the dependent variable
# is explained by the independent variable.
#
# R² ranges from 0 to 1.
#
# Higher R² generally means the model explains more variation
# in the dependent variable.


# Q7. Why is statistical analysis important before applying
# machine learning algorithms?
#
# Answer:
# Statistical analysis helps understand the data, identify
# relationships, detect unusual values, test assumptions and
# select useful features before machine learning.
# This can help in building more reliable models.


# Q8. What assumptions should be satisfied before performing
# a t-test or ANOVA?
#
# Answer:
# Important assumptions include:
#
# 1. Observations should be independent.
# 2. Data should be approximately normally distributed
#    within groups.
# 3. Variances should be reasonably similar between groups.
# 4. The dependent variable should generally be numerical.


# Q9. How can regression analysis help organizations in
# forecasting and decision-making?
#
# Answer:
# Regression analysis identifies relationships between variables
# and can be used to predict future values. Organizations can
# use it for forecasting sales, revenue, demand, prices and
# other business measures.


# Q10. Give two real-world applications where hypothesis
# testing is commonly used in data analytics.
#
# Answer:
#
# 1. A company can test whether a new advertising campaign
#    significantly increases sales.
#
# 2. A company can test whether two groups of customers have
#    significantly different average spending.


# ============================================================
# END OF EXPERIMENT 3
# ============================================================
```

```
Libraries imported successfully!
======================================================================
Dataset loaded successfully!
======================================================================

FIRST 5 ROWS:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
DATASET SHAPE:
(891, 12)

COLUMN NAMES:
['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']

DATASET INFORMATION:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

Missing values after treatment:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64

MEAN:
Age     29.361582
Fare    32.204208
dtype: float64

MEDIAN:
Age     28.0000
Fare    14.4542
dtype: float64

MODE:
Age     28.00
```